PROYECTO FINAL · INFRAESTRUCTURA III

DynamoDB, estado y bitácora auditable

B3 explicado como un profesor: qué se construyó, por qué y cómo comprobarlo

DocuProcess Cloud

Rama `feat/B3-modulo-DynamoDB`

11 de noviembre de 2026

Guía de aprendizaje + informe de cierre

Contenido

1. El problema: dónde encaja B3 en DocuProcess Cloud

2. La decisión de diseño: dos tablas, no una

3. DynamoDB desde cero (lo que hay que entender de verdad)

4. El módulo de Terraform, línea por línea

5. El código de la API: del clic a la transacción

6. Cómo se verificó: pruebas, límites y evidencia

7. Despliegue en AWS, paso a paso

8. Troubleshooting: los errores que vas a ver

9. Preguntas de examen (con respuesta)

10. Glosario y checklist de cierre

1. El problema: dónde encaja B3

DocuProcess Cloud es una aplicación para subir documentos, procesarlos de forma asíncrona y consultar su estado. El enunciado divide el sistema en cuatro piezas: un frontend estático en S3 + CloudFront, una API REST, un worker que procesa archivos y un conjunto de servicios gestionados (S3, DynamoDB, SNS, CloudWatch, CloudTrail). La infraestructura vive en una VPC con subredes públicas (ALB) y privadas (aplicación y datos).

La regla del proyecto es que **cada motor de base de datos resuelve un problema distinto**:

Necesidad	Motor	Por qué
Cuentas, correo único, contraseñas	RDS MySQL	Relaciones, `UNIQUE(email)`, transacciones ACID y consultas por atributo.
Estado del documento y cola del worker	DynamoDB `documents`	Acceso por clave, índices para usuario y estado, escala horizontal.
Historial auditable de estados	DynamoDB `document_events`	Eventos inmutables, ordenados y consultables por documento, usuario o día.
Archivos originales	S3	Binarios; no tienen sentido dentro de una base de datos.

La tarea **B3** es la encargada del segundo y tercer renglón. Su objetivo original era crear la tabla `documents`, que guarda el estado de cada documento y funciona como cola del worker. Durante el diseño se añadió una segunda tabla, `document_events`, para que el «historial» que pide el enunciado fuera una bitácora real y no un campo que se sobrescribe.

> **Idea clave:** una cosa es el *estado actual* (¿en qué va este documento?) y otra el *registro de lo que pasó* (¿quién lo cambió, cuándo y por qué?). Mezclarlos en un solo ítem es cómodo al principio y problemático cuando hay que auditar.

1.1. Qué debía entregar B3

- Un módulo de Terraform `infra/modules/dynamodb/` con la tabla `documents`.
- PK `documentId`, dos GSIs (`userId-createdAt` y `status-createdAt`), modo on-demand, cifrado y point-in-time recovery.
- Una política de recurso que diera acceso al rol preexistente de las instancias, porque el Sandbox no permite crear roles ni adjuntar políticas IAM.
- Un esquema idéntico al que se usa en DynamoDB Local para desarrollo.
- Evidencia: `describe-table`, un `put-item` visible en el GSI de estado y un `update-item` condicional que falla la segunda vez.

2. La decisión de diseño: dos tablas, no una

La primera tentación es guardar el historial como una lista dentro del documento:

```
{
  "documentId": "...",
  "status": "DONE",
  "events": [
    { "status": "PENDING", "at": "..." },
    { "status": "PROCESSING", "at": "..." },
    { "status": "DONE", "at": "..." }
  ]
}
```

Funciona en una demo, pero tiene cuatro problemas serios:

1. **Límite de 400 KB por ítem.** El historial crece sin límite; el documento dejaría de caber.
2. **Reescritura completa.** Cada transición reescribe toda la lista, no solo el evento nuevo.
3. **Consultas incómodas.** «Todos los cambios de un día» obligaría a leer todos los documentos.
4. **Historial frágil.** Un bug puede borrar la lista; no hay inmutabilidad real.

La alternativa elegida es una tabla append-only separada. Cada transición produce un ítem nuevo, con clave `(documentId, eventAt)`. El estado actual sigue en `documents` para que la API y el worker lean rápido; la historia vive en `document_events`.

Criterio	Lista dentro del documento	Tabla de eventos
Crecimiento	Limitado por 400 KB	Independiente y horizontal
Escritura	Reescribe todo	Un `PutItem` O(1)
Consulta por día/usuario	Requiere leer todos los documentos	GSIs dedicados
Inmutabilidad	Solo por código	La política no permite Update/Delete
Costo	Más escritura por transición	Una escritura pequeña por evento

> **Regla de oro:** el estado es una proyección; los eventos son la verdad. Si algún día hay que reconstruir el estado, la bitácora permite hacerlo.

2.1. Por qué el alta es transaccional

Crear un documento implica dos escrituras: el ítem en `documents` y el evento inicial `PENDING` en `document_events`. Si se hicieran por separado y fallara la segunda, quedaría un documento sin rastro de su creación. `TransactWriteItems` garantiza que **o se escriben las dos o no se escribe ninguna**.

Además, la condición del documento (`attribute_not_exists(documentId)`) hace que el alta sea idempotente frente a duplicados: si el `documentId` ya existe, la transacción se cancela en vez de sobrescribir.

3. DynamoDB desde cero

3.1. Tabla, ítem y atributos

Una tabla de DynamoDB es una colección de ítems. Un ítem es un conjunto de atributos, parecido a una fila JSON. No hay esquema fijo: solo se declaran los atributos que forman parte de una clave. Por eso en Terraform se ven bloques `attribute` únicamente para `documentId`, `userId`, `status`, `createdAt`, etc.

3.2. Clave de partición y clave de ordenación

La clave primaria decide cómo se distribuyen los datos:

- **Partición (hash):** determina en qué partición cae el ítem. En `documents` es `documentId`.
- **Ordenación (range):** ordena los ítems dentro de una partición. En `document_events` es `eventAt`.

Con una clave compuesta `(documentId, eventAt)`, todos los eventos de un documento viven juntos y se leen en orden cronológico con una sola `Query`. Eso es exactamente lo que necesita una bitácora.

3.3. Índices secundarios globales (GSI)

Un GSI es una «vista» de la tabla con otra clave. Permite consultar por un atributo que no es la clave principal sin hacer `Scan`:

Tabla	GSI	Consulta que resuelve
`documents`	`userId-createdAt`	Documentos de un usuario, más recientes primero.
`documents`	`status-createdAt`	Cola: los `PENDING` más antiguos primero.
`document_events`	`userId-eventAt`	Auditoría de un usuario entre documentos.
`document_events`	`day-eventAt`	Auditoría global de un día sin `Scan`.

Proyección ALL significa que el índice copia todos los atributos del ítem. Cuesta más almacenamiento, pero evita una segunda lectura a la tabla base. Para este proyecto, con ítems pequeños, es la opción correcta.

3.4. Capacidad on-demand

`PAY_PER_REQUEST` cobra por lectura y escritura real, sin aprovisionar RCU/WCU. En una demo intermitente es lo sensato: no se paga capacidad ociosa ni hay que ajustar alarmas de throttling. `PROVISIONED` tendría sentido con carga predecible y sostenida.

3.5. Consistencia

Las lecturas de la tabla base pueden ser eventualmente consistentes (por defecto) o fuertemente consistentes con `ConsistentRead: true`. Los GSIs son **siempre** eventualmente consistentes. Por eso la API lee el detalle con `ConsistentRead`, pero la lista por usuario usa el GSI y acepta el pequeño retraso de replicación.

3.6. Palabras reservadas y condiciones

`status` es una palabra reservada de DynamoDB. En una expresión hay que usar un alias:

```
ConditionExpression: '#s = :pending'
ExpressionAttributeNames: { '#s': 'status' }
ExpressionAttributeValues: { ':pending': 'PENDING' }
```

Las condiciones son la base de la concurrencia optimista: dos workers pueden intentar reclamar el mismo documento; solo el primero que cumpla `#s = :pending` gana, el otro recibe `ConditionalCheckFailedException`.

3.7. Transacciones

`TransactWriteItems` agrupa hasta 100 operaciones (o 4 MB) y las confirma todas o ninguna. Si una condición falla, el SDK no lanza `ConditionalCheckFailedException` (eso es para una operación suelta), sino `TransactionCanceledException` con un arreglo `CancellationReasons`. En este proyecto se comprobó:

```
TransactionCanceledException
CancellationReasons: [ConditionalCheckFailed, None]
```

El primer renglón es la condición del documento; el segundo, la del evento, que no falló. Y el evento no se escribió: atomicidad real.

4. El módulo de Terraform, línea por línea

El módulo vive en `infra/modules/dynamodb/` y expone cuatro variables:

Variable	Default	Para qué
`table_name`	`documents`	Tabla de estado y cola.
`events_table_name`	`document_events`	Bitácora append-only.
`app_role_arn`	—	Rol de las instancias que recibe la política de recurso.
`enable_point_in_time_recovery`	`true`	Fallback si el Sandbox deniega PITR.

4.1. La tabla `documents`

```
resource "aws_dynamodb_table" "documents" {
  name         = var.table_name
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "documentId"

  attribute { name = "documentId" type = "S" }
  attribute { name = "userId"     type = "S" }
  attribute { name = "status"     type = "S" }
  attribute { name = "createdAt"  type = "S" }

  global_secondary_index {
    name            = "userId-createdAt"
    hash_key        = "userId"
    range_key       = "createdAt"
    projection_type = "ALL"
  }
  global_secondary_index {
    name            = "status-createdAt"
    hash_key        = "status"
    range_key       = "createdAt"
    projection_type = "ALL"
  }

  server_side_encryption { enabled = true }
  point_in_time_recovery { enabled = var.enable_point_in_time_recovery }
  deletion_protection_enabled = false
}
```

Tres detalles que suelen preguntarse:

- **¿Por qué `createdAt` es string y no número?** Porque el contrato C1 fija ISO-8601 UTC con milisegundos. Ese formato ordena lexicográficamente igual que cronológicamente, y es legible en la consola.
- **¿Por qué no se declara `filename`?** Porque no es clave. DynamoDB no necesita declarar todos los atributos; de hecho, declarar de más confunde y no aporta.

- **¿Por qué `deletion_protection_enabled = false`?** Es un laboratorio que se destruye y recrea cada día; la protección impediría `terraform destroy`.

4.2. La tabla `document_events`

Su clave es compuesta: `documentId` (hash) + `eventAt` (range). El `eventAt` no es solo un timestamp: es `ISO-8601 con milisegundos + "#" + 6 hex`. El sufijo aleatorio evita que dos eventos del mismo milisegundo colisionen y se sobrescriban.

Además de `userId`, se guarda `eventDay` (`YYYY-MM-DD` UTC) para el GSI `day-eventAt`. ¿Por qué no usar directamente una clave de día calculada al consultar? Porque DynamoDB no permite funciones en las claves: el valor tiene que estar materializado en el ítem.

4.3. Cifrado y respaldo

`server_side_encryption { enabled = true }` sin `kms_key_arn` usa la llave administrada por AWS (`aws/dynamodb`). El Sandbox no permite crear KMS keys propias. `point_in_time_recovery` permite restaurar a un segundo de los últimos 35 días; se controla con variable porque la SCP podría denegarlo.

4.4. La política de recurso

El rol `AmazonSSMRoleForInstancesQuickSetup` solo trae permisos de SSM. El Sandbox no permite `CreateRole` ni adjuntar políticas, así que la única vía es una **política basada en el recurso**: la tabla declara quién puede hacer qué.

```
resource "aws_dynamodb_resource_policy" "documents" {
  resource_arn = aws_dynamodb_table.documents.arn

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Sid    = "AppDocumentsTableAndIndexes"
      Effect = "Allow"
      Principal = { AWS = var.app_role_arn }
      Action = [
        "dynamodb:GetItem", "dynamodb:PutItem",
        "dynamodb:UpdateItem", "dynamodb:Query",
        "dynamodb:TransactWriteItems"
      ]
      Resource = [
        aws_dynamodb_table.documents.arn,
        "${aws_dynamodb_table.documents.arn}/index/*"
      ]
    }]
  })
}
```

El `Resource` incluye la tabla y sus índices: sin el segundo elemento, una `Query` al GSI fallaría con `AccessDeniedException`. Este detalle se probó en el spike A1 y es una de las causas más comunes de error.

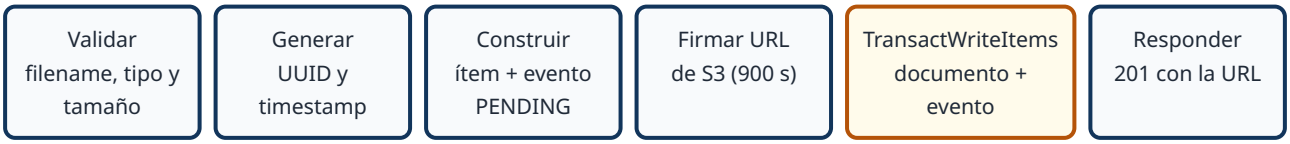
> **Riesgo real:** si la SCP deniega `dynamodb:PutResourcePolicy`, no hay plan B sin IAM propio. No se puede «inventar» un rol en el Sandbox. La política de la tabla de eventos es igual, pero sin `UpdateItem` ni `DeleteItem`: la app no puede reescribir la historia ni siquiera con un bug.

5. El código de la API: del clic a la transacción

5.1. Configuración que falla rápido

`config.ts` exige `DDB_EVENTS_TABLE` y parsea `DDB_TRANSACTIONS` como booleano estricto (`true` / `false`). Si falta una variable obligatoria, la API no arranca: es mejor fallar al inicio que descubrir el problema con el primer documento.

5.2. Alta de documento



El evento inicial tiene `actor: "api"`, `toStatus: "PENDING"`, sin `fromStatus` (no hay estado anterior) y `schemaVersion: 1`. El `eventAt` se construye con el mismo timestamp del documento más un sufijo aleatorio.

5.3. Condiciones y atomicidad

```
const documentPut = {
  TableName: config.ddbTable,
  Item: item,
  ConditionExpression: 'attribute_not_exists(documentId)',
};
const eventPut = {
  TableName: config.ddbEventsTable,
  Item: event,
  ConditionExpression: 'attribute_not_exists(eventAt)',
};

if (config.ddbTransactions) {
  await ddb.send(new TransactWriteCommand({
    TransactItems: [{ Put: documentPut }, { Put: eventPut }],
  }));
  return;
}

await ddb.send(new PutCommand(documentPut));
await ddb.send(new PutCommand(eventPut));
```

La condición del evento protege **su clave exacta**. Es un error común pensar que `attribute_not_exists(documentId)` impide que un documento tenga más de un evento: con clave compuesta, un `eventAt` distinto es un ítem distinto y la condición se evalúa contra ese ítem. Se corrigió a `attribute_not_exists(eventAt)` y se documentó.

5.4. Fallback del Sandbox

Si la SCP deniega `TransactWriteItems`, `DDB_TRANSACTIONS=false` escribe primero el documento y después el evento. El sistema sigue operando, pero una caída entre ambas escrituras deja un documento sin evento `PENDING`. Es una deuda consciente, no un comportamiento ideal: se documenta y se detecta comparando estado con bitácora.

5.5. Endpoint de bitácora

`GET /api/documents/{id}/events` hace tres cosas en orden:

1. Valida que el id sea UUID; si no, 404 (no se filtra si existe o no).
2. Lee el documento y comprueba que pertenezca al usuario autenticado. Si no existe o es ajeno, 404. No se usa 403 para no revelar la existencia de documentos de otros.
3. Consulta `document_events` en orden ascendente, límite 200, y responde `{ items: [...] }`.

Si DynamoDB falla, 503 `storage_unavailable`. Los errores se registran en el log con contexto, sin exponer detalles internos al cliente.

6. Cómo se verificó

6.1. Pruebas unitarias

Los tests usan dobles en memoria (`FakeDb`, `FakeAws`). No necesitan Docker ni credenciales y verifican la lógica de la API: validaciones, 404/401/503, orden de la bitácora y respuesta de OpenAPI. Resultado: **67 pruebas en verde**.

> **Lo que un test unitario NO prueba:** que la política de recurso funcione, que la tabla exista, que la transacción real se comporte igual o que el GSI esté ACTIVE. Para eso está DynamoDB Local y, en última instancia, AWS.

6.2. DynamoDB Local

El `docker-compose.yml` levanta `amazon/dynamodb-local` en el puerto 8001. Se crearon las dos tablas con los mismos comandos del contrato C1 y se comprobó:

Comprobación	Resultado
`describe-table` de ambas tablas	**ACTIVE, PAY_PER_REQUEST, 2 GSIs ACTIVE**
`put-item` PENDING + query `status-createdAt`	**Devuelve el ítem**
Update condicional dos veces	**Segundo falla con `ConditionalCheckFailedException`**
Transacción documento + evento	**Ambos escritos**
Transacción con condición fallida	**Nada escrito; `TransactionCanceledException`**
GSIs de eventos	**Consultas por usuario y por día devuelven el evento**

6.3. Terraform y ShellCheck

`terraform fmt -recursive`, `init -backend=false` y `validate` pasan. El `plan` no pudo completarse por el desajuste de cuenta (ver capítulo 7). El userData se renderizó en sus dos variantes (provisional y con artefacto) y pasó ShellCheck.

7. Despliegue en AWS, paso a paso

7.1. Requisitos

- Credenciales de la cuenta correcta del Sandbox en `~/.aws/credentials`.
- `infra/terraform.tfvars` a partir de `terraform.tfvars.example`, con `alert_email`, `db_password` y `jwt_secret`.
- Estado de Terraform de esa misma cuenta. Si el estado es de otra, no se debe aplicar.

7.2. Comandos

```
cd infra
terraform init
terraform fmt -check -recursive
terraform validate
terraform plan -out=tfplan
terraform apply tfplan

terraform output dynamodb_table_name
terraform output dynamodb_events_table_name
```

7.3. Verificación contra AWS

```
aws dynamodb describe-table --table-name documents \
  --query 'Table.{Status:TableStatus,Billing:BillingModeSummary.BillingMode,
                  GSIs:GlobalSecondaryIndexes[].{Name:IndexName,Status:IndexStatus}}'

aws dynamodb put-item --table-name documents --item '{
  "documentId":{"S":"11111111-1111-4111-8111-111111111111"},
  "userId":{"S":"7"},"status":{"S":"PENDING"},
  "createdAt":{"S":"2026-11-11T14:00:00.000Z"}}'

aws dynamodb query --table-name documents --index-name status-createdAt \
  --key-condition-expression '#s = :p' \
  --expression-attribute-names '{"#s":"status"}' \
  --expression-attribute-values '{":p":{"S":"PENDING"}}'
```

El segundo `update-item` con `--condition-expression '#s = :pending'` debe fallar con `ConditionalCheckFailedException`. Esa es la prueba de la concurrencia optimista que usará el worker.

7.4. El bloqueo encontrado

En este entorno, `aws sts get-caller-identity` devuelve la cuenta **556880009270**, mientras el estado local de Terraform pertenece a **491699813584**. Además, no existe `terraform.tfvars`, la SCP deniega `ec2:DescribeImages` y el instance profile del rol de SSM no está en la cuenta actual. Por eso **no se ejecutó apply**: hacerlo mezclaría recursos de dos cuentas.

> **Cómo se arregla:** copiar las credenciales de la cuenta correcta, verificar con `aws sts get-caller-identity`, crear el `tfvars` y, si el estado es de otra cuenta, empezar con un estado limpio (`terraform state rm` o borrar el `terraform.tfstate` local tras respaldarlo) antes de planificar.

8. Troubleshooting

Síntoma	Causa probable	Solución
`AccessDeniedException` al consultar un GSI	La política no incluye `${table_arn}/index/*`	Añadir el ARN de índices al `Resource`.
`ValidationException: Reserved keyword`	`status` sin alias	Usar `#s` en `ExpressionAttributeNames`.
`ConditionalCheckFailedException`	Otro proceso cambió el estado	Esperado en la cola: reintentar o descartar el documento.
`TransactionCanceledException`	Una condición de la transacción falló	Revisar `CancellationReasons`; no es un fallo de red.
`ResourceInUseException`	La tabla ya existe (state perdido)	Importar con `terraform import` o borrar la tabla si es de laboratorio.
PITR denegado por SCP	El Sandbox no permite la operación	`enable_point_in_time_recovery=false`.
`plan` falla en `data.aws_ami`	SCP deniega `ec2:DescribeImages`	Usar una AMI/instancia permitida o la cuenta correcta.
Estado de otra cuenta	State local de una sesión anterior	No aplicar; limpiar/importar y verificar `account_id`.

9. Preguntas de examen

1. ¿Por qué no se usa RDS para los documentos?

Porque el patrón de acceso es por clave y por índices conocidos, el esquema varía (`result` depende del tipo), el historial crece sin límite y la carga del worker debe aislarse de la autenticación. RDS brilla con relaciones y unicidad; DynamoDB, con clave-valor a escala.

2. ¿Por qué dos tablas y no una?

Para separar estado actual de historia inmutable, evitar el límite de 400 KB, permitir consultas por usuario/día y poder negar `UpdateItem` / `DeleteItem` en la bitácora.

3. ¿Qué pasa si dos workers reclaman el mismo documento?

Ambos hacen un `UpdateItem` condicionado a `#s = :pending`. Solo uno cumple la condición; el otro recibe `ConditionalCheckFailedException` y sigue con el siguiente.

4. ¿Por qué `eventAt` lleva un sufijo aleatorio?

Porque la clave de ordenación debe ser única por documento. Dos eventos en el mismo milisegundo colisionarían y uno sobrescribiría al otro; el sufijo de 6 hex lo evita.

5. ¿Por qué `eventDay` existe si se puede calcular del timestamp?

Porque DynamoDB no permite funciones en las claves de un GSI: el valor debe estar materializado en el ítem. `eventDay` habilita el GSI `day-eventAt`.

6. ¿Qué diferencia hay entre `ConditionalCheckFailedException` y `TransactionCanceledException`?

El primero es de una operación suelta (`UpdateItem`, `PutItem`). El segundo es de una transacción; trae `CancellationReasons` con el motivo por cada ítem.

7. ¿Por qué la política de recurso y no un rol IAM?

Porque el Sandbox no permite crear roles ni adjuntar políticas. La política de recurso la declara el propio recurso (DynamoDB) y nombra al rol preexistente como principal.

8. ¿Qué garantiza `PAY_PER_REQUEST`?

Cobro por petición y escalado automático de capacidad; no garantiza consistencia fuerte ni orden global. La consistencia se elige por lectura y el orden se diseña con la clave.

9. ¿Qué pasa si falla la escritura del evento con `DDB_TRANSACTIONS=false`?

Queda un documento sin evento `PENDING`. La API devuelve error, pero el documento ya existe; es la deuda documentada del fallback. Con transacciones, no ocurre.

10. ¿Por qué el endpoint devuelve 404 y no 403 para un documento ajeno?

Para no revelar que el documento existe. Un atacante no puede enumerar ids ajenos distinguishedo entre «no existe» y «existe pero no es tuyo».

10. Glosario y checklist

10.1. Glosario

Término	Significado
Ítem	Unidad de datos de DynamoDB; conjunto de atributos identificado por una clave.
GSI	Índice secundario global: otra clave para consultar la misma tabla.
Proyección	Atributos copiados al índice. `ALL` copia todos.
On-demand	Capacidad automática; se paga por petición.
PITR	Point-in-time recovery: restaurar a un segundo de los últimos 35 días.
Append-only	Solo se agregan ítems; no se actualizan ni se borran.
Concurrencia optimista	Actualizar solo si el estado sigue siendo el esperado.
SCP	Service Control Policy: límite de permisos de la organización del Sandbox.
SSE	Cifrado en reposo del lado del servidor.

10.2. Checklist de cierre

- **Hecho** Módulo con dos tablas, 4 GSIs y 2 políticas.
- **Hecho** Variables, outputs y userData conectados.
- **Hecho** Alta transaccional y endpoint de bitácora.
- **Hecho** 67 tests, typecheck, build, validate y ShellCheck.
- **Hecho** Verificación funcional en DynamoDB Local.
- **Hecho** Contrato, comparativa y trazabilidad actualizados.
- **Pendiente** `terraform apply` en la cuenta correcta del Sandbox.
- **Pendiente** Worker (B5/B6) para generar transiciones reales.

> **Para llevarte una sola idea:** DynamoDB no es «una base de datos SQL más simple». Es un motor que se diseña desde las consultas. En B3 las consultas eran claras —por documento, por usuario, por estado, por día— y por eso el modelo quedó pequeño: dos tablas, cuatro índices y condiciones bien puestas.